Software Configuration management plan

PROJECT: Travel Advisor Website

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

TABLE OF CONTENTS

1 Identification **3**
1.1 Document overview ***3***
1.2 References ***3***
1.2.1 Project References 3
1.3 Conventions ***3***
2 Organization **3**
2.1 Configuration management principles ***4***
2.2 Configuration management in a development cycle ***4***
2.3 Configuration management of releases ***4***
2.4 Configuration management of prototypes *Error! Bookmark not defined.*
2.5 Tasks in development and maintenance ***5***
2.6 Archiving versions ***5***
2.7 Link with bugs and features ***5***
3 Configuration identification **7**
3.1 Identification rules of configuration items ***7***
3.2 Identification rules of documents ***7***
4 Document History **7**

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

1 Identification

This document amplifies the "Configuration management" of the Project Management Plan.

1.1 Document overview

This document is the Software Configuration Management Plan (CMP) for the QA-Project. It defines how all software artifacts — source code, documents, third-party libraries, and release packages — are identified, controlled, and tracked throughout the project lifecycle.

1.2 References

1.2.1 Project References

#	Document Identifier	Document Title
[R1]	ID	PROJECT MANAGEMENT PLAN

1.3 Conventions

- Branch names: lowercase — e.g., "feature/QAPROJ-[JIRATicketNumber]-login-module"
- Software versions: "Vm.n.p" format — e.g., "V1.2.3"
- Document revisions: "Rev.XXX" format — e.g., "Rev.001"
- Baseline / archive tags: uppercase with hyphens — "BL-QB-V1.0.0"
- All branch names and commit messages must include the Jira ticket ID

Commented [Aa1]: What's the project name?

Commented [Aa2R1]: QB

2 Organization

The CMP is managed by the SCM Manager with support from all project developers. Responsibilities are distributed as follows:

Role	Person	Responsibility
SCM Manager	TBD	Maintains repository structure, enforces branching rules, approves merges to **'main'**, takes baselines, manages releases.
Project Manager	Ahmed Nabil	Approves major version releases and scope changes.
Developer (×5)	Team members	Creates feature branches, commits code with Jira references, submits PRs for review

Commented [Aa3]: Whose role is it?

Commented [Aa4R3]: Ahmed Nabil

Commented [Aa5]: He approves merges, but project manager approves major version release and scope changes. Is this right? Did I capture the difference?

also, how does the project manager cooperate with the SCM Manager in baseline and releases?

Commented [Aa6R5]: The SCM Manager and PM are the same person

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

2.1 Configuration management principles

Configuration management is performed using *Git* hosted on *GitHub* (private repository, accessible to team members only). *GitHub* integrates with *Jira* for task and bug tracking, ensuring every code change is traceable to a Jira ticket.

GitHub repository architecture:

Branch	Purpose
'main'	Production-ready code. Tagged on every official release and baseline
'develop'	Integration branch for ongoing development. When ready for release, merged directly into 'main'.
'feature/<TICKET-ID>-description'	New features, branched from 'develop' branch. Merged back via PR after review
'hotfix/<TICKET-ID>-description'	Emergency production fixes, branched from 'main'.

Modifications between branches are managed through Pull Requests (PRs), which require at least one peer review before merging.

2.2 Configuration management in a development cycle

During each sprint, developers work on `**feature/<TICKET-ID>-description**` branches created from `**develop**`.

2.2.1 Sprint workflow

1. At sprint start, developers create their `feature/` branch from the latest `develop`.
2. Code is committed with Jira ticket references in commit messages.
3. When work is complete, a Pull Request is submitted targeting `develop`.
4. The PR is reviewed by at least one peer; the SCM Manager reviews structural or high-risk changes.
5. Approved PRs are merged into `develop`.
6. At sprint end, the SCM Manager tags the current state of `main` as a *Baseline*

2.3 Configuration management of releases

When all features planned for a release are merged into `develop` and the sprint is complete, `develop` is merged directly into `main` — no intermediate release branch is used.

Release workflow:

1. All planned features are merged into `develop` via reviewed PRs.
2. The SCM Manager opens a PR from `develop` → `main`.
3. The PR is reviewed and signed off by the Project Manager.
4. `develop` is merged into `main`.
5. SCM Manager tags the merge commit on `main` as a baseline (`BL-QB-Vm.n.p`).
6. A *GitHub Release* is published with release notes, built artifacts, and SHA256 checksum.

For hotfixes on an already-released version:
- A `hotfix/<TICKET-ID>-description` branch is created from `main`.
- After fixing and reviewing, it is merged into both `main` (tagged as a new patch) and `develop`.

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

2.4 Tasks in development and maintenance

The SCM Manager and developers perform the following operations during the software lifecycle:

Event	Operation
Setting up the project	SCM Manager creates repository structure and configures branch protection rules on GitHub
Starting a new sprint	SCM Manager confirms `develop` is up to date; creates Jira sprint board
Starting a new feature	Developer creates `feature/<TICKET-ID>-description` branch from `develop`
Completing a feature	PR submitted, peer-reviewed, merged into `develop`; Jira ticket closed
Delivering a release	SCM Manager opens PR from `develop` → `main`; Project Manager signs off; merged, tagged (`BL-QA-Vm.n.p`), and published as GitHub Release
Fixing a production bug	Developer creates `hotfix/` branch from `main`; merged to `main` + `develop` after review; new patch tag created

Developer locations for code changes:

Type of Modification	Branch
New feature for next release	`feature/` branch from `develop`
Emergency fix in production	`hotfix/` branch from `main`

2.5 Archiving versions

Each release is archived on *GitHub Releases* (accessible at the project's GitHub repository page). Each archived release contains:

- Source code snapshot (Git tag)
- Built artifacts / deliverables (if applicable)
- Release notes / changelog

Release assets are retained indefinitely in GitHub. The SCM Manager is responsible for ensuring each GitHub Release is properly published after merging to `main`.

2.6 Link with bugs and features (under modification)

Commented [Aa7]: How are we gonna link with JIRA?

Traceability between code changes and Jira tickets is enforced as follows:

- Branch naming: Every branch name must include the Jira ticket ID.
- Example: `feature/QAPROJ-42-login-module`, `hotfix/QAPROJ-55-null-pointer`
- Commit messages: Every commit message must start with the Jira ticket ID.
- Example: `QAPROJ-42: Add login module`
- Pull Requests: Every PR description must reference the corresponding Jira ticket.
- -Jira status flow: `To Do` → `In Progress` (branch created) → `In Review` (PR opened) → `Done` (PR merged)

This ensures it is always possible to trace which code changes belong to which feature or bug fix.

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

2.7 Baseline management

A baseline is a formally approved snapshot of the repository at a specific milestone. Once set, a baseline can only be modified through the project's change control process.

2.7.1 Approach: Single Baseline from `main`

The QB-Project uses a single, unified baseline type, taken from the `main` branch after every significant merge event (end of sprint, release, or hotfix).

2.7.2 When a baseline is taken

Trigger Event	Branch Tagged
Sprint delivery merged into `main`	`main`
Release merged into `main`	`main`
Hotfix merged into `main`	`main`

2.7.3 Who takes the baseline

The SCM Manager, **Ahmed Nabil**, is responsible for creating and pushing all baselines. No additional approval is required for routine sprint baselines; release baselines require sign-off from the Project Manager before the merge to `main` proceeds.

2.7.4 How a baseline is created

1. The merge to `main` is completed and verified.
2. SCM Manager creates a Git tag on the merge commit in `main` using the convention `BL-QB-Vm.n.p` (e.g., `BL-QB-V1.0.0`).
3. Tag is pushed to the remote GitHub repository.
4. An entry is recorded in the Baseline Register below.

Baseline Register:

Baseline Name	Date	Git Tag	Event	Taken By	Notes
BL-QB-V1.0.0	4/4/2026	Initial release	Sprint Finish	Ahmed Nabil	First sprint delivery

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

3 Configuration identification

3.1 Identification rules of configuration items

Software configuration items are versioned using the following format:
<ProjectName> _Vm.n.p
where:

- "Vm " is the major version of the configuration item,
- n is the minor version number,
- p is the incremental minor version number, if necessary.

The version number of the configuration item Vm.n.p starts at V1.0.0.

The number "m" of major version is incremented when substantial modifications are made to the device, like when **updating of the intended use**, or **Adding new modules / functionalities requiring new regulatory submission**.

The number "n" of minor version is incremented when non-substantial modifications are made to the device, like when **adding new functionalities to existing modules**, or **updating the GUI**.

The number "p" of incremental minor version is incremented when minor modifications are made to the device, like when **modifying existing functionalities**, **minor update of the GUI**.

Example: `QBProject_V1.2.3` — versioning starts at **V1.0.0**

All configuration items under management for the QA-Project:

CI Type	Examples	Location
Source Code	Application code, test scripts, config files	GitHub repository
Documents	CMP, Test Plans, Requirements, User Stories	GitHub `/docs` folder or shared drive

3.2 Identification rules of documents

Project documents are identified using the following format:

QA-<document-number> Rev.<revision-index>

Segment	Description
QB	Project acronym
<document-number>	Document type and sequential number — e.g., `CMP-001`, `TP-001`
Rev.<revision-index>	Approved revision starting at `Rev.001`, incrementing with each approved change

Example: `QB-CMP-001 Rev.001`

The term "revision" is used for documents (instead of "version") to avoid confusion with software version numbers.

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License

4 Document History

Baseline Name	Date	Git Tag	Event
Rev.01	4/4/2026	QB-CMP-001 REV.01	Initial release

This Template is the property of Cyrille Michaud
License terms: see http://blog.cm-dm.com/post/2011/11/04/License